# 第 14 章：用 dataclass 与 Pydantic 管理结构化数据

一句话概括本章：**内部可信状态重在表达和维护，外部不可信数据重在验证；不要把所有结构都塞进 `dict`，也不要把所有结构都做成 Pydantic 模型。**

Agent 程序会不断搬运结构化数据：模型生成 tool 参数，HTTP 或数据库返回记录，工具再把结果交回 Agent 循环；与此同时，程序还要保存运行阶段、重试次数和事件列表。它们看起来都能用 `dict` 表示，但承担的责任不同。

本章所说的"数据模型"是指**用类或类型声明描述一组结构化数据**，不是 Python 语言中由 `__iter__`、`__enter__` 等特殊方法组成的广义对象模型。

学完本章，你应该能够：

- 分清 `dict`、`TypedDict`、`dataclass` 和 Pydantic `BaseModel` 的职责；
- 解释 dataclass 自动生成了什么，以及类型注解为什么不会自动校验；
- 分清必填、可缺失、可为 `None`、未知字段和默认值；
- 为 tool 参数选择合适的类型转换与严格模式；
- 验证嵌套输入，保留准确的错误路径，并正确序列化时间等值；
- 用 JSON Schema 描述 tool 参数，同时说清它不能保证什么。

## 14.1 先建立 Java → Python 心智模型

这四种结构不是从低级到高级的升级链，而是解决四类不同问题。

| Python 结构 | 可以先类比为 | 运行时形态 | 常规构造/验证入口会校验 | 适合放在哪里 |
| --- | --- | --- | --- | --- |
| `dict` | `Map<String, Object>` | 可变映射 | 否 | 键本来就动态的临时数据 |
| `TypedDict` | 为固定键 Map 补充的静态接口 | 仍然是普通 `dict` | 否 | 已可信、仍需按字典协议传递的数据 |
| `dataclass` | 精简版 POJO、Lombok 数据类 | 自定义类实例 | 否 | 进程内状态、配置快照、值对象 |
| `BaseModel` | DTO + 数据绑定 + Bean Validation + Schema | Pydantic | 是 | JSON、LLM、HTTP、 |

n/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f89d3201bdd3285df23e02467478.html

求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计费

| | | | | |
|---|---|---|---|---|
| `BaseModel` | DTO + 数据绑定 + Bean Validation + Schema 生成 | Pydantic 模型实例 | 是 | JSON、LLM、HTTP、数据库等 I/O 边界 |

这些类比都不能完全等同。

- `TypedDict` 不会生成一个可在运行时识别的新容器类型，也不会像 Jackson 反序列化那样检查输入。
- dataclass 虽然会生成构造、显示和比较方法，但不会像 Jakarta Bean Validation 那样根据注解自动拒绝错误值。
- `BaseModel` 把解析、转换、约束和错误收集集中在模型入口完成；其默认转换规则、缺失语义和验证时机也不等同于 Java DTO。

选择时先问数据来自哪里：，

1. 键集合本来就是开放的，用 `dict` 。
2. 键固定、数据已经可信、消费者必须拿到字典，用 `TypedDict` 。
3. 数据由当前进程维护，主要需要属性访问、默认值、显示和比较，用 dataclass。
4. 数据刚从进程边界进入，需要拒绝、转换、报错或生成 schema，用 `BaseModel` 。

实际项目经常组合使用：先用 `BaseModel` 验证 tool 参数，再用 dataclass 保存执行状态，最后用另一个 `BaseModel` 校验并序列化工具结果。

## 14.2 dict 与 TypedDict：容器没变，静态信息变了

当键由运行时数据决定时，字典就是准确的模型：

```
counts_by_tool: dict[str, int] = {}
counts_by_tool[tool_name] = counts_by_tool.get(tool_name, 0) + 1
```

这里无法提前列出所有 `tool_name` ，强行声明固定字段反而会误导读者。

如果键已经固定，但调用方仍要求字典协议，可以使用 `TypedDict` ：

```
from typing import TypedDict


class ToolSummary(TypedDict):
    request_id: str
    item_count: int
```

wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f89d3201bdd3285df23e02467478.html

.相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

```
summary: ToolSummary = {"request_id": "req-1", "item_count": 2}
assert type(summary) is dict
```

Pyright 可以据此发现漏键、错键和值类型不匹配，IDE 也能补全键名。但运行时的 `summary` 仍是 `dict`。把 `json.loads()` 的结果直接标注成 `ToolSummary`，只是告诉检查器"请相信它"，并没有验证任何数据。

因此，`TypedDict` 最适合描述**已经经过边界校验的内部字典**。外部 JSON 仍应先验证，不能靠类型标注把愿望变成事实。

参考：Python `TypedDict` 文档。

## 14.3 dataclass：为可信内部对象减少样板代码

`@dataclass` 是标准库类装饰器。Python 执行类定义时，装饰器读取带注解的字段，并按配置生成常用方法。默认情况下，最值得关注的是：

- `__init__()`：接收初始化参数并保存字段；
- `__repr__()`：生成便于调试的对象表示；
- `__eq__()`：按声明顺序比较两个同类实例的字段值；
- 字段元数据：可以通过 `dataclasses.fields()` 读取。

```
from dataclasses import dataclass, field
from typing import Literal


@dataclass
class RunState:
    run_id: str
    phase: Literal["created", "done"] = "created"
    attempts: int = 0
    events: list[str] = field(default_factory=list)
    debug_context: dict[str, str] = field(default_factory=dict, repr=False)
```

`run_id` 没有默认值，因此构造时必须传入，并且要放在有默认值的字段之前。下面这些行为都来自 dataclass 生成的方法：

```
first = RunState(run_id="run-1")
second = RunState(run_id="run-2")
```

```
first.events.append("validated")
assert first.events == ["validated"]
assert second.events == []
assert first != second
```

这可以先类比为 Java 中用 Lombok 减少 POJO 的构造器、 `toString()` 和 `equals()` 样板代码，但有三个重要边界。

## 注解不会触发运行时校验

```
state = RunState(run_id=123)  # 静态检查器会报告，普通 Python 运行时仍会创建实例
assert state.run_id == 123
```

生成的 `__init__()` 只负责赋值，不会因为 `run_id: str` 自动调用 `isinstance()` ，也不会把 `123` 转成字符串。可信内部代码配合 Pyright 通常已经够用；从 LLM 或 HTTP 收到的字典不能直接展开成 dataclass 来冒充校验。

`Literal` 在这里同样主要服务于静态检查。普通 dataclass 不会在运行时拒绝其他字符串。

## 可变默认值必须按实例创建

每个 `RunState` 都应该拥有独立的事件列表。 `field(default_factory=list)` 保存的是创建默认值的函数；每次构造实例且没有传 `events` 时，生成的 `__init__()` 都会调用一次 `list()` 。

不可变默认值可以直接写成 `attempts: int = 0` 。可变值不要写成 `events: list[str] = []` ；它表达共享对象的危险语义，现代 dataclass 也会在定义类时拒绝这种未哈希的默认值。

## repr、比较和安全不是一回事

`repr=False` 只是不让字段出现在自动生成的 `repr` 中。 `debug_context` 仍然可以访问，默认也仍参与相等比较，所以它不能用来保护秘密。

如果缓存等字段不应决定逻辑相等性，可以显式使用 `field(compare=False)` 。是否排除应来自对象语义，不能只为得到期望的测试结果。

## frozen 只是浅层禁止重新赋值

```
@dataclass(frozen=True)
class FrozenPlan:
    name: str
    tags: list[str] = field(default_factory=list)
```

```python
@dataclass(frozen=True)
class FrozenPlan:
    name: str
    tags: list[str] = field(default_factory=list)


plan = FrozenPlan("daily", ["agent"])
# plan.name = "weekly"      # FrozenInstanceError
plan.tags.append("search")  # 合法：修改 list，没有重新绑定 plan.tags
```

`frozen=True` 可以先类比为只暴露只读字段的值对象，但它不会递归冻结字段指向的对象。真正要求标签不可变时，应考虑 `tuple[str, ...]`，并继续检查更深层元素是否可变。

参考：Python `dataclasses` 文档。

## 14.4 BaseModel：先验证边界，再让核心代码相信类型

Pydantic `BaseModel` 也使用类型注解声明字段，但目标不是少写 `__init__()`，而是把不可信输入变成经过验证的对象。

可以先把它类比为 Java 中“Jackson 绑定 DTO + Bean Validation 校验 + OpenAPI Schema 生成”的组合，但不能完全等同：Pydantic 的解析和校验由模型入口统一触发，并且默认允许一部分类型转换。

```python
from pydantic import BaseModel, ConfigDict, Field


class SearchArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")

    query: str = Field(min_length=1, max_length=100)
    limit: int = Field(default=3, strict=True, ge=1, le=5)


args = SearchArgs.model_validate({"query": "Python", "limit": 2})
assert args.query == "Python"
assert args.limit == 2
```

```
不可信 dict / JSON 文本
          |
          | 解析类型、应用约束、运行 validator
          ▼
   SearchArgs 实例
          |
          ▼
只接收可信模型的核心业务函数
```

本章统一使用 Pydantic v2 的两个显式入口：

```
# 已经有 Python 对象，通常是 dict
args = SearchArgs.model_validate(python_data)

# 手上还是 str 或 bytes 形式的 JSON
args = SearchArgs.model_validate_json(raw_json)
```

`model_validate_json()` 同时解析 JSON 文本并验证字段。JSON 语法错误、字段类型错误和业务 validator 失败都会以 `ValidationError` 报告。已经有 Python 对象时直接使用 `model_validate()`，不必先序列化再解析。

## 14.5 必填、可空、默认值和未知字段互不等价

Java DTO 常把“字段没传”和“字段值为 null”都表现成 `null`。Pydantic v2 会明确区分这两种输入状态。

| 字段声明 | 键能否缺失 | 值能否为 `None` | 键缺失时的结果 |
| --- | --- | --- | --- |
| `name: str` | 否 | 否 | `missing` 错误 |
| `name: str = "guest"` | 是 | 否 | 使用 `"guest"` |
| `name: str \| None` | 否 | 是 | `missing` 错误 |
| `name: str \| None = None` | 是 | 是 | 使用 `None` |

`T | None` 只表示合法值集合中包含 `None`，不会自动提供默认值。**可空由类型决定，可缺失由默认值决定。**

未知字段又是独立问题。Pydantic 默认采用 `extra="ignore"`，未声明的键可能被静默丢弃。对 tool 参数来说，这会让拼错的 `limti` 消失，然后悄悄启用 `limit` 的默认值。

```
class SearchArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")

    query: str
    limit: int = 3
```

`extra="forbid"` 会把未知键变成 `extra_forbidden` 错误。是否使用它要由协议兼容策略决定：封闭 tool 参数通常适合禁止未知字段，开放元数据容器则可能需要 `ignore` 或 `allow`。

## 14.6 类型转换与 strict：协议接受什么，必须主动决定

Pydantic 默认偏向把"可以理解的输入"转换成目标类型。这对表单和环境变量很方便，但 tool 协议通常需要更精确的边界。

以 `int` 为例：

| 输入 | 默认宽松 `int` | `strict=True` 的 `int` |
| --- | --- | --- |
| `"3"` | 转成 `3` | 拒绝 |
| `True` | 转成 `1` | 拒绝 |
| `None` | 拒绝 | 拒绝 |
| 键缺失 | 无默认值则拒绝 | 无默认值则拒绝 |

Python 中 `bool` 是 `int` 的子类，所以手写代码里的 `isinstance(True, int)` 会得到 `True`。如果业务要求"必须是真正的整数"，应明确写：

```
limit: int = Field(default=3, strict=True, ge=1, le=5)
```

严格策略可以放在字段、模型配置或一次验证调用上：

```
from pydantic import BaseModel


class ConversionProbe(BaseModel):
    count: int
```

ers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f89d3201bdd3285df23e02467478.html

喽的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

```
ConversionProbe.model_validate({"count": "3"}, strict=True)
```

不要机械地给所有字段开启全局 strict。JSON 没有 `datetime` 原生类型，时间通常需要从 ISO 8601 字符串解析；JSON 本来就有 number 和 boolean，整数和布尔字段则更适合按协议严格检查。Python 对象输入与 JSON 输入在少数严格模式规则上也可能不同，应使用真实边界数据测试。

此外，默认值通常不会因为"模型定义成功"就自动经历与输入相同的验证。模型作者应保证默认值本身正确；确实需要校验默认值时，再显式了解 `validate_default` 。

参考：Strict Mode 与 Conversion Table。

## 14.7 嵌套模型：让结构、转换和错误路径一起递归

嵌套对象直接声明为另一个模型类型：

```
from datetime import datetime
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field


class SearchFilters(BaseModel):
    model_config = ConfigDict(extra="forbid")

    language: Literal["zh", "en"] = "zh"
    published_after: datetime | None = None
    keywords: list[str] = Field(default_factory=list)
```

外层 `SearchArgs.filters` 收到字典后，Pydantic 会递归生成 `SearchFilters` 实例。JSON 中的时间字符串可以被解析成 `datetime` ，列表中的每个元素也会按 `str` 检查。失败位置因此可以精确到 `filters.keywords.1` ，而不是只剩一句"参数错误"。

这里还有一个容易被忽略的边界： `datetime` 只说明值是日期时间，不自动保证它带时区。当前项目输入使用带 `Z` 的 UTC 时间，因此能与目录中的 UTC 时间比较；生产协议如果禁止无时区时间，还应增加明确约束，例如使用 Pydantic 的 `AwareDatetime` 或自定义验证。

在 Pydantic 模型里， `Literal["zh", "en"]` 不再只是静态提示。运行时验证会拒绝其他值，相应枚举也会进入 JSON Schema。这一点与普通 dataclass 中的 `Literal` 不同。

ers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f89d3201bdd3285df23e02467478.html

庚的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下

# 14.8 validator：只表达类型和 Field 覆盖不了的规则

先使用字段类型、 `Literal` 和 `Field` 表达机械规则。只有确实需要规范化或跨字段关系时，再编写 validator。

## 字段 validator 处理单个值

`min_length=1` 能拒绝空字符串，却不能拒绝只含空白的字符串。字段 validator 可以先去掉首尾空白，再检查结果：

```
from pydantic import BaseModel, Field, field_validator


class Query(BaseModel):
    text: str = Field(min_length=1)

    @field_validator("text")
    @classmethod
    def normalize_text(cls, value: str) -> str:
        normalized = value.strip()
        if not normalized:
            raise ValueError("text 不能只包含空白")
        return normalized
```

默认的字段 validator 在基础类型和 `Field` 约束通过后运行。它的**返回值才是后续真正保存的值**；忘记 `return` 会隐式返回 `None` 。

装饰器在这里不是仅供框架扫描的 Java 注解。Python 会先按照装饰器语义包装函数，Pydantic 再把它登记进模型的验证流程。

## 模型 validator 处理字段之间的不变量

```
from datetime import datetime
from typing import Literal, Self

from pydantic import BaseModel, model_validator


class SearchWindow(BaseModel):
    published_after: datetime | None = None
    sort: Literal["relevance", "newest"] = "relevance"
```

```python
from datetime import datetime
from typing import Literal, Self

from pydantic import BaseModel, model_validator


class SearchWindow(BaseModel):
    published_after: datetime | None = None
    sort: Literal["relevance", "newest"] = "relevance"

    @model_validator(mode="after")
    def require_newest_sort(self) -> Self:
        if self.published_after is not None and self.sort != "newest":
            raise ValueError("指定时间过滤时必须按最新排序")
        return self
```

`mode="after"` 表示字段和嵌套模型准备好后，再检查完整实例。实例方法检查完必须返回要继续使用的模型；没有修改时返回 `self`。

本章 `ToolResult` 还用模型 validator 保证：成功结果不能同时带 `error`，失败结果必须提供错误说明。注意它只保证代码明确写出的关系，并没有自动保证“失败时 items 一定为空”等未声明规则。

自定义 validator 是普通 Python 代码，越多就越需要针对顺序、错误和边界值单独测试。其任意逻辑也未必能完整表达进 JSON Schema。

参考：Pydantic Validators。

## 14.9 ValidationError：保留结构化错误，不要压成一句话

下面的输入同时包含严格整数、枚举、时间、列表元素和未知字段错误：

```json
{
  "query": "Python",
  "limit": true,
  "filters": {
    "language": "fr",
    "published_after": "昨天",
```

```
{
  "query": "Python",
  "limit": true,
  "filters": {
    "language": "fr",
    "published_after": "昨天",
    "keywords": ["agent", 7]
  }
}
```

`ValidationError.errors()` 返回结构化错误列表。每项最常用的字段是：

- `loc`：从根模型走到错误值的路径，如 `("filters", "keywords", 1)`；
- `msg`：适合展示给人的说明；
- `type`：适合程序识别的错误类别，如 `int_type`、`missing`；
- `input`：导致失败的原始值。

可以把位置格式化为：

```
filters.keywords.1: Input should be a valid string (string_type)
```

API 层可以再把这些细节映射成自己的错误协议，但不要捕获后只返回“参数错误”，也不要把 `str(error)` 当作唯一机器接口。记录 `input` 前要考虑密码、token、个人信息和大文本泄漏。

与常见手写校验相比，Pydantic 还能在一次验证中收集多项独立字段错误；但模型级 validator 是否执行，仍取决于前置字段是否已经成功形成可用模型。

参考：Pydantic Error Handling。

## 14.10 验证发生在何时：不要让可信对象再次失真

`model_validate()`、`model_validate_json()` 和常规模型构造会执行验证。但“曾经验证通过”不代表对象此后永远合法。

Pydantic 模型默认允许属性重新赋值，而且默认不会校验赋入的新值：

```
args = SearchArgs.model_validate({"query": "Python", "limit": 2})
args.limit = "many"  # 默认不触发赋值校验，静态检查器会报告
```

工程上可以选择：

- 把边界模型当作验证后的快照，不再原地改字段；
- 使用 `model_copy()` 创建新对象，但要了解 `update` 数据默认不会重新验证；
- 确实需要可变且每次赋值都检查时，使用 `ConfigDict(validate_assignment=True)`；
- 需要不可变接口时，再评估 `ConfigDict(frozen=True)` 及其浅层边界。

`model_construct()` 会跳过常规验证，只适合已经完全可信且清楚其语义的高级场景，不能用来接收外部数据。

这与 Java 中“DTO 曾经通过 Bean Validation，后来 setter 又写入非法值”的问题类似：验证保证的是某个时刻和入口，不是对象终身不变量。

## 14.11 序列化：Python 字典、JSON 兼容字典与 JSON 文本

“得到字典”和“得到 JSON 文本”是不同操作。

| 调用 | 返回类型 | `datetime` 等值 | 适合场景 |
| --- | --- | --- | --- |
| `model_dump()` | `dict` | 仍是 Python `datetime` | Python 代码继续计算或断言 |
| `model_dump(mode="json")` | `dict` | 变成 ISO 8601 `str` | 仍想操作字典，但值需兼容 JSON |
| `model_dump_json()` | `str` | 已编码进 JSON 文本 | 直接传输或保存 |

```
from datetime import datetime

from pydantic import BaseModel


class Event(BaseModel):
    happened_at: datetime


event = Event.model_validate({"happened_at": "2026-01-01T00:00:00Z"})
python_data = event.model_dump()
json_ready = event.model_dump(mode="json")
json_text = event.model_dump_json()

assert isinstance(python_data["happened_at"], datetime)
assert json_ready["happened_at"] == "2026-01-01T00:00:00Z"
```

```
assert isinstance(python_data["happened_at"], datetime)
assert json_ready["happened_at"] == "2026-01-01T00:00:00Z"
assert isinstance(json_text, str)
```

直接把包含 `datetime` 的 `model_dump()` 结果交给标准库 `json.dumps()`，通常会得到 `TypeError`。方法名里有 `dump` 不代表所有嵌套值都已经 JSON 化。

本章为工具输出单独定义 `SearchItem` 和 `ToolResult`。这使"本地工具 → Agent 循环"也成为受控边界：嵌套结果先形成一致结构，再通过 `model_dump_json()` 递归编码。

序列化只改变表示，不会重新证明现实事实。格式正确的 URL 可能打不开，合法时间也可能是上游捏造的。

参考：Pydantic Serialization。

## 14.12 JSON Schema：描述 tool 参数，不替代服务端验证

```
schema = SearchArgs.model_json_schema()
```

返回值是可 JSON 化的 Python `dict`。本章模型生成的 schema 包含：

- 顶层 object、`properties` 和必填字段；
- `query` 的长度与说明；
- `limit` 的默认值、整数类型和 `1..5` 上下界；
- `sort`、`language` 的枚举；
- `SearchFilters` 的嵌套定义；
- `additionalProperties: false` 表达的未知字段禁止策略。

这些信息可以放入支持 JSON Schema 的 tool/function 参数描述，让模型知道应该生成什么形状。它相当于给调用者协议说明，而不是给服务端免检通行证。

Schema 不能保证：

1. LLM 一定生成符合 schema 的参数；
2. 未写入模型的业务规则自动成立；
3. 任意 Python validator 都能完整转换成声明式 schema；
4. URL、标题、时间和数字对应真实、最新且有权限的数据；
5. 数据库唯一性、库存、鉴权和其他外部状态成立。

因此，服务端仍必须执行 Pydantic 验证。结构校验只是业务正确性的第一层，不是全部。

README × README × readme × +
ate/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f89d3201bdd3285df23e02467478.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

因此，服务端仍必须执行 Pydantic 验证。结构校验只是业务正确性的第一层，不是全部。

参考：Pydantic JSON Schema。

## 14.13 把四种结构放进一条 tool 数据流

本章示例把职责拆成三个核心对象：

- `SearchArgs(BaseModel)`：接收不可信工具参数，禁止未知字段，严格校验整数和布尔值；
- `ToolResult(BaseModel)`：约束工具输出，递归包含 `SearchItem`，并检查 `ok` 与 `error` 的关系；
- `RunState(dataclass)`：保存当前进程自己维护的运行阶段和事件，不验证外部输入。

完整数据流如下：




输入中的 `filters` 最初是字典，`published_after` 最初是字符串。验证成功后：

- `args.filters` 是 `SearchFilters`；
- `published_after` 是 `datetime`；
- `query` 的首尾空白已被字段 validator 去掉；
- `limit`、布尔值、枚举、未知字段和跨字段关系都已经检查；
- 核心函数不再重复解析原始字典键。

示例中的 `local_search()` 是确定性的本地替身，只读取两个固定条目，不联网、不需要 API Key。它只实际使用时间过滤、条数和是否包含摘要；`language` 与 `keywords` 用来演示嵌套验证，并没有实现真实搜索语义。

这个替身能证明本章的数据流、校验、状态更新和序列化，但不能证明真实 HTTP 场景中的 DNS、超时、状态码、限流、重试、取消、鉴权和内容准确性。

## 与第 10 章手写校验的迁移对照

示例还复刻了第 10 章的一组用户规则，验证迁移到模型后没有悄悄放宽协议。

| 手写规则 | Pydantic 声明 |
| --- | --- |
| `id` 必须是真正的正整数 | `Field(strict=True, gt=0)` |
| `age` 必须是 `0..150` 的真正整数 | `Field(strict=True, ge=0, le=150)` |
| `active` 必须是真正的 bool | `bool` 字段使用 `Field(strict=True)` |
| 字段集合必须精确匹配 | 必填字段 + `ConfigDict(extra="forbid")` |
| `name` 不能只含空白 | 长度约束 + `field_validator` |

Pydantic 的优势是声明集中、嵌套路径统一、schema 和序列化可以复用。手写校验在规则极少或协议高度特殊时仍然合理。迁移的关键不是“换了更高级的库”，而是保持原业务接受和拒绝的集合不变。

# 14.14 Java 工程师最容易误判的地方

1. **把注解当运行时约束。** 普通函数、`TypedDict` 和 dataclass 的类型注解主要服务于静态检查；外部数据必须真正走验证入口。
2. **把 `T | None` 当成可缺失。** 没有默认值时仍然必填。
3. **忘记 Pydantic 默认忽略未知字段。** 封闭协议应明确考虑 `extra="forbid"` 。
4. **把自动转换当成总是友好。** `"3"` 和 `True` 被转成整数可能改变协议语义。
5. **给所有字段一刀切 strict。** 时间等 JSON 不原生支持的类型本来就需要解析。
6. **忘记 validator 必须返回值。** 字段 validator 返回要保存的值，after 模型 validator 返回模型。
7. **把验证过一次理解成永久合法。** 默认属性赋值和某些高级构造/复制入口可能绕开再次验证。
8. **混淆三种 dump 结果。** `model_dump()` 的字典可能仍含 Python 专用对象。
9. **把 `frozen` 当深度不可变。** 它不递归冻结 list、dict 等子对象。
10. **把 schema 当安全证明。** 它既不执行服务端校验，也不验证权限、数据库状态和现实事实。

# 14.15 运行本章示例

本章包含三个文件：

- `readme.md` ： 独立教程、选择标准、边界与练习；
- `example.py` ： 五组确定性实验和完整本地 tool 数据流；

ate/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f89d3201bdd3285df23e02467478.html

近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

## 14.15 运行本章示例

本章包含三个文件：

- `readme.md`：独立教程、选择标准、边界与练习；
- `example.py`：五组确定性实验和完整本地 tool 数据流；
- `requirements.txt`：Pydantic v2 运行依赖。

仓库要求 Python `>=3.14`。先在**仓库根目录**安装本章依赖：

```
.venv/bin/python -m pip install \
  -r java_to_python/chapters/14_data_models/requirements.txt
```

再运行示例：

```
.venv/bin/python -B java_to_python/chapters/14_data_models/example.py
```

`-B` 只是不生成 `__pycache__`，不改变程序语义。现有 `run_all.py` 只覆盖前 10 章，因此本章使用独立命令。

建议按五段输出观察：

1. `TypedDict` 的运行时类型仍是 `dict`，dataclass 自动生成显示和比较行为；
2. 缺失、`None`、未知字段、宽松转换和严格模式得到不同结果；
3. 手写校验与 Pydantic 对合法/非法用户记录做出相同决定；
4. Python 模式字典、JSON 模式字典和 JSON 文本中的时间类型不同；
5. 完整 pipeline 更新 `RunState`，输出 `ToolResult`，并打印 schema 摘要与嵌套错误路径。

从仓库根目录执行静态检查：

```
.venv/bin/ruff check java_to_python/chapters/14_data_models
.venv/bin/ruff format --check java_to_python/chapters/14_data_models
.venv/bin/pyright --pythonpath .venv/bin/python \
  java_to_python/chapters/14_data_models/example.py
```

本次教程改写已在 CPython 3.14.7、Pydantic 2.13.5、Ruff 0.16.5 和 Pyright 1.1.411 下运行并检查。版本约束仍以仓库 `pyproject.toml` 和本章 `requirements.txt` 为准。

## 14.16 递进练习

# 14.16 递进练习

## 练习 1：扩展内部状态

给 `RunState` 增加 `warnings: list[str]` 和不进入 `repr` 的 `trace_context`。创建两个实例，只修改第一个，验证列表没有共享、`repr` 没有泄漏上下文，并解释为什么这里使用 dataclass。

## 练习 2：画出存在性与严格模式矩阵

给 `SearchArgs` 增加一个注释字段，分别尝试"必填可空"和"可缺失可空"。再测试 `limit` 的 `"3"`、`True`、`None` 和缺失，记录每份输入的接受或拒绝结果以及错误类型。

## 练习 3：约束嵌套域名

给 `SearchFilters` 增加 `domains: list[str]`。去除每项首尾空白并拒绝空项，确认错误路径能定位到 `filters.domains.1`，同时观察生成的 schema。

## 练习 4：补齐失败结果

让 `local_search()` 在查询词为 `"offline"` 时返回结构合法的失败 `ToolResult`。验证正常结果和受控失败都能序列化，而 `ok=False, error=None` 会被模型 validator 拒绝。

## 练习 5：恢复批量唯一性规则

为 `list[UserModel]` 设计批模型，增加"用户 ID 不重复"的集合级验证。验证重复 ID 得到明确错误，`id=True` 先被严格整数规则拒绝，并解释为什么 JSON Schema 不能证明数据库中的全局唯一性。

# 14.17 本章小结

- `dict` 适合真正动态的键；`TypedDict` 为可信字典增加静态形状，不增加运行时验证。
- dataclass 适合可信内部对象，自动生成样板方法，但类型注解不负责运行时检查。
- `BaseModel` 适合 I/O 边界，负责解析、约束、错误定位、序列化与 schema。
- 必填、可空、默认值和未知字段是独立维度；宽松转换与 strict 是协议决策。
- 嵌套模型让数据转换和错误路径一起递归；validator 只补充声明式字段规则无法表达的逻辑。
- 验证只对实际经过的入口和时刻负责；赋值、绕过验证的 API、外部状态和现实事实仍需单独治理。
- JSON Schema 能指导 tool 调用，但服务端必须继续验证，业务层仍要检查权限、唯一性和真实性。

```python
"""第 14 章：为 Agent 的工具输入、输出和内部状态选择数据模型。"""

# 阅读顺序：先看四类数据结构的声明，再看中间的校验/搜索函数，最后从 main() 进入五组实验。
# 完整数据流是：JSON 文本 -> SearchArgs 边界校验 -> 本地工具 -> ToolResult -> JSON 文本；
# RunState 则只记录进程内部状态，不负责校验外部输入。
#
# 标准库分别负责 JSON 文本、轻量数据类、带时区时间与静态类型描述。
import json
from dataclasses import FrozenInstanceError, dataclass, field, fields
from datetime import UTC, datetime
from typing import Literal, Self, TypedDict

# Pydantic 是本章的运行时边界工具：BaseModel 声明模型，Field 补充字段规则，
# validator 表达普通字段类型无法覆盖的规则，ValidationError 汇总失败细节。
from pydantic import (
    BaseModel,
    ConfigDict,
    Field,
    ValidationError,
    field_validator,
    model_validator,
)


def print_section(title: str) -> None:
    """让终端输出能和 README 的实验小节逐段对应。"""
    # f-string 会把 title 当前指向的字符串嵌入结果；开头的 \n 先留一行，分隔相邻实验。
    print(f"\n=== {title} ===")


# TypedDict 只为类型检查器描述“这个 dict 应有哪些键及值类型”，不会创建新的容器类，
# 也不会在运行时检查数据。后面的 summary 仍由普通字典字面量构造并按字典协议使用。
class ToolSummary(TypedDict):
    """描述已经在程序内部形成、仍以 dict 传递的固定结构。"""

    # 三个键都没有使用 NotRequired，因此静态约定要求它们同时存在。
    request_id: str
```

```
33  class ToolSummary(TypedDict):
36      # 三个键都没有使用 NotRequired，因此静态约定要求它们同时存在。
37      request_id: str
38      item_count: int
39      cached: bool
40
41
42  # @dataclass 会在执行类定义时读取这些带注解字段，并生成 __init__、__repr__ 和 __eq__。
43  # 可以类比减少 Java POJO 的样板代码，但字段注解仍不会自动变成运行时类型检查。
44  @dataclass
45  class RunState:
46      """进程内部的轻量可变状态：这里不承担不可信输入的验证。"""
47
48      # 无默认值的字段是必传初始化参数，并且必须放在下面有默认值的字段之前。
49      run_id: str
50      # Literal 把静态允许值收窄为三个字符串；普通 dataclass 运行时不会据此拒绝其他值。
51      phase: Literal["created", "validated", "completed"] = "created"
52      attempts: int = 0
53      # default_factory 保存 list 这个“创建函数”；每次缺省构造 RunState 时都会调用一次 list()。
54      # 因而每个实例都拿到独立列表；直接写 [] 会表达共享可变默认值，并被 dataclass 拒绝。
55      events: list[str] = field(default_factory=list)
56      # repr=False 只把调试上下文从自动 __repr__ 隐藏；它仍能访问，也仍参与默认的相等比较。
57      # 这只是减少日志噪声，并不是秘密字段的安全保护。
58      debug_context: dict[str, str] = field(default_factory=dict, repr=False)
59
60
61  # frozen=True 让 dataclass 生成阻止字段重新绑定/删除的行为；下面会分别尝试“重新绑定 name”
62  # 与“修改 tags 指向的 list”，借此观察冻结只作用于实例属性这一层。
63  @dataclass(frozen=True)
64  class FrozenPlan:
65      """frozen 只阻止字段重新绑定，不会递归冻结字段指向的对象。"""
66
67      name: str
68      tags: list[str] = field(default_factory=list)
69
70
```

```python
# BaseModel 与 dataclass 的关键区别是：构造/验证时会真正执行运行时解析和约束。
# 这个小模型刻意把“必填”“有默认值”“允许 None”“禁止未知键”拆开观察。
class PresenceModel(BaseModel):
    """只用于观察必填、默认值、可空和未知字段四条独立规则。"""

    # extra="forbid" 表示输入中只要出现未声明的键就报错，避免拼错键后被静默忽略。
    model_config = ConfigDict(extra="forbid")

    # 没有 = 默认值，所以 required_name 与 required_nullable 都必须在输入里出现。
    required_name: str
    # “str | None”只把 None 加入允许值集合，不代表这个键可以缺失。
    required_nullable: str | None
    # 既允许 None 又以 None 为默认值：键可以缺失，缺失时保存 None。
    optional_nullable: str | None = None
    # 类型不含 None，但有默认值：键可缺失，显式传 None 仍会失败。
    retry_count: int = 3


# 此处不设置模型级 strict，便于稍后把同一份输入分别交给宽松模式和 strict=True。
class ConversionProbe(BaseModel):
    """同一字段分别以默认宽松模式和调用级 strict 模式验证。"""

    count: int


# 该模型把第 10 章的手写用户规则声明为 Pydantic 约束，后面的成对实验会验证：
# 换成模型之后，bool 冒充 int、数字字符串转换和额外字段仍然都不会被放行。
class UserModel(BaseModel):
    """保留第 10 章手写规则中的严格整数与未知字段语义。"""

    model_config = ConfigDict(extra="forbid")

    # strict=True 禁止 Pydantic 把 True 或字符串转换成整数；gt/ge/le 再限制数值范围。
    id: int = Field(strict=True, gt=0)
    # min_length=1 能拒绝空串，却不能拒绝 "   "，所以还需要下面的字段验证器。
    name: str = Field(strict=True, min_length=1)
```

```python
# 该模型把第 10 章的手写用户规则声明为 Pydantic 约束，后面的成对实验会验证：
# 换成模型之后，bool 冒充 int、数字字符串转换和额外字段仍然都不会被放行。
class UserModel(BaseModel):
    """保留第 10 章手写规则中的严格整数与未知字段语义。"""

    model_config = ConfigDict(extra="forbid")

    # strict=True 禁止 Pydantic 把 True 或字符串转换成整数；gt/ge/le 再限制数值范围。
    id: int = Field(strict=True, gt=0)
    # min_length=1 能拒绝空串，却不能拒绝 "   "，所以还需要下面的字段验证器。
    name: str = Field(strict=True, min_length=1)
    age: int = Field(strict=True, ge=0, le=150)
    # bool 也使用 strict=True，明确拒绝 0、1、"true" 等可转换但协议不接受的输入。
    active: bool = Field(strict=True)
    note: str = Field(strict=True)

    # 装饰器从紧邻函数的 @classmethod 开始向上应用：先把函数包装为类方法，
    # 再由 field_validator 把它登记到`name 字段的验证流程中。这不是仅供阅读的 Java 注解。
    @field_validator("name")
    @classmethod
    def name_must_not_be_blank(cls, value: str) -> str:
        """长度约束拦不住纯空白文本，所以补上这条业务规则。"""
        # strip() 产生移除首尾空白的新字符串；这里只用它判断，不改变最终保存的原始 name。
        if not value.strip():
            raise ValueError("name 不能只包含空白")
        # validator 的返回值才会继续流入模型；忘记 return 会隐式返回 None。
        return value


# SearchFilters 会作为 SearchArgs.filters 的字段类型。外层收到嵌套 dict 后，Pydantic 会
# 递归验证并把它转换成 SearchFilters 实例，错误位置也会保留 filters 这一层路径。
class SearchFilters(BaseModel):
    """嵌套筛选条件也拥有自己的未知字段策略和约束。"""

    model_config = ConfigDict(extra="forbid")
```

```python
class SearchFilters(BaseModel):

    # Literal 在 Pydantic 中既参与运行时校验，也会进入生成的 JSON Schema 枚举信息。
    language: Literal["zh", "en"] = "zh"
    # Python 输入可直接给 datetime；JSON 没有时间类型，因此 ISO 8601 字符串会在边界解析。
    published_after: datetime | None = None
    # 每次缺失 keywords 时创建新列表，避免不同模型实例共享后续可变状态。
    keywords: list[str] = Field(default_factory=list)


# SearchArgs 位于 Agent/tool 的不可信输入边界：类型、范围、未知字段和跨字段关系都在这里集中。
class SearchArgs(BaseModel):
    """处在 Agent/tool 边界上的输入模型。"""

    model_config = ConfigDict(extra="forbid")

    # Field 给 str 增加长度、描述等约束；description 主要供生成 schema 和文档使用。
    query: str = Field(
        min_length=1,
        max_length=100,
        description="要检索的非空查询词；首尾空白会被移除",
    )
    # 工具明确只接受真正的 JSON 整数，不能让 "3" 或 true 偷偷变成 3/1；
    # default=3 只处理键缺失，ge/le 让合法范围闭合在 1..5。
    limit: int = Field(default=3, strict=True, ge=1, le=5, description="最多返回几条结果")
    # JSON 自带 boolean 类型，因此这里要求调用者真的发送 true/false，不接受常见替代写法。
    include_snippets: bool = Field(
        default=True,
        strict=True,
        description="是否在结果中包含摘要",
    )
    sort: Literal["relevance", "newest"] = "relevance"
    # default_factory=SearchFilters 表示 filters 缺失时现场构造一个带自身默认值的嵌套模型。
    filters: SearchFilters = Field(default_factory=SearchFilters)

    # 默认字段验证器在 query 的基础类型和 Field 约束通过后运行，用返回值完成规范化。
    @field_validator("query")
```

```python
class SearchArgs(BaseModel):
    # 默认字段验证器在 query 的基础类型和 Field 约束通过后运行，用返回值完成规范化。
    @field_validator("query")
    @classmethod
    def normalize_query(cls, value: str) -> str:
        """字段验证器接收当前字段值，并必须返回后续真正保存的值。"""
        # str 不可变，strip() 返回新对象；原始输入字典/JSON 文本不会被原地修改。
        normalized: str = value.strip()
        # 长度约束已排除 ""，这里继续排除 strip 后才变空的纯空白字符串。
        if not normalized:
            raise ValueError("query 不能只包含空白")
        return normalized

    # mode="after" 表示所有字段及嵌套 filters 都准备好后，再在完整实例上检查字段关系。
    @model_validator(mode="after")
    def published_filter_requires_newest_sort(self) -> Self:
        """模型验证器在字段就绪后检查跨字段业务不变量。"""
        # is not None 用来区分“确实指定了发布时间”与默认的“未启用时间过滤”。
        if self.filters.published_after is not None and self.sort != "newest":
            raise ValueError("指定 published_after 时 sort 必须是 'newest'")
        # after 模式必须返回要继续使用的模型；这里未改实例，所以原样返回 self。
        return self


# 工具输出也要建模：每个目录条目先成为 SearchItem，ToolResult 再以它作为嵌套元素类型。
class SearchItem(BaseModel):
    """工具结果中的一条嵌套记录。"""

    model_config = ConfigDict(extra="forbid")

    title: str
    url: str
    published_at: datetime
    # 有 = None 才表示字段既可显式为 None，也可在输入中完全缺失。
    snippet: str | None = None

```

```python
# ToolResult 是本地工具到 Agent 循环的输出边界。模型校验让后面的序列化始终面对一致结构。
class ToolResult(BaseModel):
    """可安全序列化回 Agent 循环的工具结果。"""

    model_config = ConfigDict(extra="forbid")

    # Literal 固定工具名；调用者不传时取 "search"，传其他名字则失败。
    tool: Literal["search"] = "search"
    ok: bool = Field(strict=True)
    # 省略 items 时为当前结果创建独立空列表；传入字典元素时也会递归生成 SearchItem。
    items: list[SearchItem] = Field(default_factory=list)
    error: str | None = None

    # 单看 ok: bool 和 error: str | None 无法表达两者必须配套，因此使用模型级验证器。
    @model_validator(mode="after")
    def success_and_error_must_agree(self) -> Self:
        """类型本身表达不了 ok 与 error 的对应关系，必须显式检查。"""
        if self.ok and self.error is not None:
            raise ValueError("成功结果不能同时包含 error")
        if not self.ok and not self.error:
            raise ValueError("失败结果必须说明 error")
        return self


# 这个常量目录在导入/运行本模块、执行到此处时立即构造；它不是每次搜索才联网获取的数据。
# 两个 published_at 都带 UTC 时区，之后才能与解析出的带时区筛选时间安全比较。
CATALOG: list[SearchItem] = [
    SearchItem(
        title="Python 数据模型实践",
        url="https://local.example/python-data-models",
        published_at=datetime(2026, 8, 20, 9, 30, tzinfo=UTC),
        snippet="用结构化边界保护工具调用。",
    ),
    SearchItem(
        title="Agent 工具参数设计",
        url="https://local.example/agent-tool-args",
```

```
    SearchItem(
        title="Agent 工具参数设计",
        url="https://local.example/agent-tool-args",
        published_at=datetime(2026, 7, 15, 8, 0, tzinfo=UTC),
        snippet="从 JSON Schema 到运行时校验。",
    ),
]

# 元组不可变，适合保存手写协议要求的精确键集合；下面会转为 set 做集合差运算。
USER_FIELDS = ("id", "name", "age", "active", "note")


def format_validation_error(error: ValidationError) -> list[str]:
    """保留每条错误的嵌套路径和类型，而不是压成一句“参数错误”。"""
    # 用 list 收集全部错误，而不是遇到第一项就抛弃后续细节；调用者可选择逐行打印或拼接。
    lines: list[str] = []
    # errors() 返回由 dict 组成的列表；include_url=False 只省略文档链接，不影响路径和类型。
    for detail in error.errors(include_url=False):
        # loc 是从根模型走到坏值的路径元组，例如 ("filters", "keywords", 1)。
        # 生成器逐个把字段名/下标转成 str，再由 ".".join(...) 得到 filters.keywords.1。
        # 模型整体 validator 的 loc 可能为空，此时 or 会选右侧的可读占位符。
        location: str = ".".join(str(part) for part in detail["loc"]) or "<model>"
        # msg 面向读者，type 更适合作为程序识别的错误类别；这里同时保留两者。
        lines.append(f"{location}: {detail['msg']} ({detail['type']})")
    return lines


def manual_validate_user(data: object) -> dict[str, object]:
    """第 10 章规则的最小只读复刻，用于和模型校验对照。"""
    # 参数先写成最宽的 object，因为边界数据可能根本不是字典；先验证容器形状再取键。
    if not isinstance(data, dict):
        raise TypeError("用户必须是对象")

    # set 只关心成员，不关心顺序。两边集合必须完全相等，才能同时排除缺键和额外键。
    expected_fields: set[Literal['id', 'name', 'age', 'acti... = set(USER_FIELDS)
    actual_fields: set[Any] = set(data)
    if actual_fields != expected_fields:
```

```python
def manual_validate_user(data: object) -> dict[str, object]:
    """第 10 章规则的最小只读复刻，用于和模型校验对照。"""
    # 参数先写成最宽的 object，因为边界数据可能根本不是字典；先验证容器形状再取键。
    if not isinstance(data, dict):
        raise TypeError("用户必须是对象")

    # set 只关心成员，不关心顺序。两边集合必须完全相等，才能同时排除缺键和额外键。
    expected_fields: set[Literal['id', 'name', 'age', 'acti... = set(USER_FIELDS)
    actual_fields: set[Any] = set(data)
    if actual_fields != expected_fields:
        # 集合差分别算出缺少/多出的键；sorted 让错误输出不受 set 遍历顺序影响。
        missing: list[Literal['id', 'name', 'age', 'act... = sorted(expected_fields - actual_fields)
        extra: list[Any] = sorted(actual_fields - expected_fields)
        raise ValueError(f"字段不符: missing={missing}, extra={extra}")

    # 只有字段集合确认无误后才逐个读取，所以这些 [] 访问不会因缺键抛出 KeyError。
    # 变量此时仍指向输入字典中的原对象，下一段再逐项检查类型和值域。
    user_id: Any = data["id"]
    name: Any = data["name"]
    age: Any = data["age"]
    active: Any = data["active"]
    note: Any = data["note"]

    # 这里刻意用 type(...) is int，而不是 isinstance(..., int)：bool 是 int 的子类，
    # isinstance(True, int) 会得到 True，但业务上布尔值不能充当用户 ID 或年龄。
    if type(user_id) is not int or user_id <= 0:
        raise ValueError("id 必须是正整数")
    # 短路 or 保证只有 name 已经是 str 时才调用 strip()，避免错误类型再触发属性异常。
    if not isinstance(name, str) or not name.strip():
        raise ValueError("name 必须是非空字符串")
    # 0 <= age <= 150 是 Python 链式比较，相当于同时满足 0 <= age 和 age <= 150。
    if type(age) is not int or not 0 <= age <= 150:
        raise ValueError("age 必须是 0 到 150 的整数")
    if type(active) is not bool:
        raise ValueError("active 必须是布尔值")
    if not isinstance(note, str):
```

```python
def manual_validate_user(data: object) -> dict[str, object]:
    if not isinstance(note, str):
        raise TypeError("note 必须是字符串")

    # 校验成功后返回一个新 dict，并按协议顺序显式放入字段；不会原地修改调用者的 data。
    return {"id": user_id, "name": name, "age": age, "active": active, "note": note}


def local_search(args: SearchArgs) -> ToolResult:
    """确定性的本地替身：演示数据流，不声称模拟真实搜索质量与网络故障。"""
    # 初始时 candidates 与 CATALOG 指向同一个 list；这里只读取它，不会修改目录常量。
    candidates: list[SearchItem] = CATALOG
    # 只有调用者真正提供了发布时间（解析结果非 None）才启用过滤。
    if args.filters.published_after is not None:
        # 列表推导式创建新 list：逐个保留发布时间大于等于下限的 SearchItem。
        # 本讲输入里的时间带 Z，解析后与目录的 UTC 时间都含时区，因此可以比较实际先后；
        # 字段类型本身没有强制时区，生产协议若允许其他格式，还需另行约束这一点。
        candidates: list[SearchItem] = [
            item for item in candidates if item.published_at >= args.filters.published_after
        ]

    # 输出列表单独创建，稍后根据 limit 截取候选项，并按 include_snippets 决定元素表示。
    selected: list[SearchItem] = []
    # candidates[: args.limit] 返回至多 limit 个元素的新列表；目录不足时不会越界报错。
    for item in candidates[: args.limit]:
        if args.include_snippets:
            # 需要摘要时可直接收集已经过 SearchItem 校验的目录元素。
            selected.append(item)
        else:
            # 不需要摘要时重新构造一个 SearchItem，把 snippet 明确设为 None；
            # 重新构造会再次经过模型验证，也不会原地改坏 CATALOG 中的共享对象。
            selected.append(
                SearchItem(
                    title=item.title,
                    url=item.url,
                    published_at=item.published_at,
                    snippet=None,
```

```python
def local_search(args: SearchArgs) -> ToolResult:
                SearchItem(
                    title=item.title,
                    url=item.url,
                    published_at=item.published_at,
                    snippet=None,
                )
            )
    # ToolResult 会检查 ok/items/error 的类型以及成功状态的不变量，再交给调用方。
    return ToolResult(ok=True, items=selected)


def run_search(raw_json: str, state: RunState) -> ToolResult:
    """走通原始 JSON → 边界校验 → 工具 → 结构化结果。"""
    # raw_json 仍是 str。model_validate_json 同时解析 JSON 并执行嵌套模型、Field 和 validator；
    # 如果失败会直接抛 ValidationError，所以下面的内部状态不会被误记为 validated。
    args: SearchArgs = SearchArgs.model_validate_json(raw_json)
    # dataclass 是可信的内部可变状态，这里用属性赋值和 list.append 原地记录阶段与事件。
    state.phase = "validated"
    state.events.append(f"validated:{args.query}")

    # local_search 只接收已经通过边界校验的 SearchArgs，而不是继续操作原始字符串/字典。
    result: ToolResult = local_search(args)
    state.phase = "completed"
    # len(result.items) 读取结构化结果的条目数，再由 f-string 形成可追踪的事件文本。
    state.events.append(f"completed:{len(result.items)}")
    return result


def demonstrate_data_structure_choices() -> None:
    """观察 TypedDict 的运行时本质，以及 dataclass 自动生成的常用行为。"""
    print_section("1. TypedDict 与 dataclass 各自负责什么")

    # 变量注解让 Pyright 知道固定键结构；等号右侧仍是普通 dict 字面量，没有运行时构造器。
    summary: ToolSummary = {"request_id": "req-001", "item_count": 2, "cached": False}
    # type(...).__name__ 取得实际类名，预期打印 dict，从运行时角度印证 TypedDict 的定位。
```

```python
def demonstrate_data_structure_choices() -> None:
    """观察 TypedDict 的运行时本质，以及 dataclass 自动生成的常用行为。"""
    print_section("1. TypedDict 与 dataclass 各自负责什么")

    # 变量注解让 Pyright 知道固定键结构；等号右侧仍是普通 dict 字面量，没有运行时构造器。
    summary: ToolSummary = {"request_id": "req-001", "item_count": 2, "cached": False}
    # type(...).__name__ 取得实际类名，预期打印 dict，从运行时角度印证 TypedDict 的定位。
    print("TypedDict 值的运行时类型:", type(summary).__name__) .

    # 三次构造都调用 dataclass 自动生成的 __init__。前两个字段值相同，用于观察 __eq__；
    # second 则用于确认 events 的默认列表是否真的按实例隔离。
    first = RunState(run_id="run-001", debug_context={"token": "不进入 repr"})
    same_values = RunState(run_id="run-001", debug_context={"token": "不进入 repr"})
    second = RunState(run_id="run-002")

    # fields(RunState) 返回字段描述对象；列表推导式依次提取 name，证明隐藏字段仍是正式字段。
    print("dataclass 字段:", [item.name for item in fields(RunState)])
    # print 会调用自动 __repr__；debug_context 因 repr=False 不显示，但依然参与下一行 eq 比较。
    print("自动 repr 隐藏 debug_context:", first)
    print("自动 eq 按字段值比较:", first == same_values)
    # append 原地修改 first 的列表。随后同时打印两份列表和 is 身份比较：
    # first 有事件、second 仍为空且 is 为 False，三项共同证明没有共享默认列表。
    first.events.append("created")
    print(
        "default_factory 没有共享列表:", first.events, second.events, first.events is second.events
    )

    # 这个忽略标记只服务于教学反例：类型检查器会报错，但运行时 dataclass 仍照单全收。
    # 必须原样保留 pyright 指令；它不改变 Python 执行，只抑制这一行预期中的静态诊断。
    invalid_state = RunState(run_id=123)  # pyright: ignore[reportArgumentType]
    # 同时观察保存的值和实际类型，预期是 123 与 int，而不是被注解自动转成 str。
    print("注解没有触发运行时校验:", invalid_state.run_id, type(invalid_state.run_id).__name__)

    # name 是冻结实例上的属性，tags 则指向一个仍可变的 list；下面分别测试两层行为。
    plan = FrozenPlan(name="daily-search", tags=["agent"])
    try:
```

```python
def demonstrate_data_structure_choices() -> None:
    try:
        # 对冻结字段重新绑定会抛 FrozenInstanceError；忽略指令保留这个故意的类型错误示例。
        plan.name = "changed"  # pyright: ignore[reportAttributeAccessIssue]
    except FrozenInstanceError:
        # 捕获预期异常后继续实验，不让整个课程脚本在这里中断。
        print("frozen 拒绝字段重新赋值: FrozenInstanceError")
    # append 修改的是 tags 指向的嵌套 list 内容，没有执行 plan.tags = ...，所以仍然合法。
    plan.tags.append("mutable-child")
    print("frozen 内的 list 仍能修改:", plan.tags)


def demonstrate_presence_and_strictness() -> None:
    """用 None、缺失、未知字段、字符串和 bool 观察不同规则。"""
    print_section("2. 必填/默认/可空，以及宽松/严格转换")

    # required_nullable 虽然必填，但显式 None 属于它的允许值；另外两个缺失字段使用默认值。
    present_none: PresenceModel = PresenceModel.model_validate(
        {"required_name": "demo", "required_nullable": None}
    )
    # model_dump() 返回普通 Python dict，便于同时观察 None、None 默认值和整数默认值。
    print("显式 None 可通过可空字段，默认值已补入:", present_none.model_dump())

    # 每个场景是（展示标签，要验证的 Python 字典），用同一循环对比两种输入错误。
    presence_cases: list[tuple[str, dict[str, object]]] = [
        # 这里是真的没有 required_nullable 键，与上面的显式 None 是两种不同输入状态。
        ("缺少仍然必填的可空字段", {"required_name": "demo"}),
        (
            "出现未知字段",
            # 两个必填字段都存在，单独加入 unexpected，以确认 extra="forbid" 生效。
            {"required_name": "demo", "required_nullable": None, "unexpected": "x"},
        ),
    ]
    # for 会把每个二元组按位置解包：label 得到说明，payload 得到当前字典。
    for label, payload in presence_cases:
        try:
            PresenceModel.model_validate(payload)
```

```
401 def demonstrate_presence_and_strictness() -> None:
422     # for 会把每个二元组按位置解包：label 得到说明，payload 得到当前字典。
423     for label, payload in presence_cases:
424         try:
425             PresenceModel.model_validate(payload)
426         except ValidationError as error:
427             # 这些都是预期反例；格式化并打印后继续下一轮，而不是终止整组实验。
428             print(label + ":", " | ".join(format_validation_error(error)))
429
430     # 四份输入覆盖“可转换字符串、bool、显式 None、缺键”，稍后每份都验证两次。
431     probe_cases: list[tuple[str, dict[str, object]]] = [
432         ('"3"', {"count": "3"}),
433         ("True", {"count": True}),
434         ("None", {"count": None}),
435         ("缺失", {}),
436     ]
437     for label, payload in probe_cases:
438         # observations 只属于当前输入，收集宽松/严格两种结果后再并排打印。
439         observations: list[str] = []
440         # 内层元组序列固定提供（展示名，strict 参数）；再次用解包得到两个局部变量。
441         for mode, strict in (("宽松", False), ("严格", True)):
442             try:
443                 # 调用级 strict 覆盖本次验证行为，但不修改 ConversionProbe 的类定义。
444                 probe: ConversionProbe = ConversionProbe.model_validate(payload, strict=strict)
445                 # !r 使用 repr 展示值，可清楚区分整数 3 与字符串 '3'；同时打印实际类型。
446                 observations.append(f"{mode}=通过({probe.count!r}, {type(probe.count).__name__})")
447             except ValidationError as error:
448                 # 模型只有 count 一个字段，因此取第一项即可展示拒绝类别。
449                 error_type: str = error.errors(include_url=False)[0]["type"]
450                 observations.append(f"{mode}=拒绝({error_type})")
451         # "; ".join(...) 把本轮两条观察连接成一个字符串，便于横向比较模式差异。
452         print(f"输入 {label}:", "; ".join(observations))
453
454
455 def demonstrate_manual_and_model_validation() -> None:
456     """证明改用模型后没有悄悄放宽第 10 章的边界规则。"""
457     print_section("3. 手写校验与 Pydantic 校验保持同一业务语义")
```

```python
def demonstrate_manual_and_model_validation() -> None:
    """证明改用模型后没有悄悄放宽第 10 章的边界规则。"""
    print_section("3. 手写校验与 Pydantic 校验保持同一业务语义")

    # 基准记录故意标为 dict[str, object]：键是字符串，但各字段值的具体类型不同。
    valid_user: dict[str, object] = {
        "id": 1,
        "name": "Alice",
        "age": 28,
        "active": True,
        "note": "普通字段",
    }
    # 每个测试场景仍是（标签，payload）二元组。字典的 | 运算会创建新字典：
    # 右侧同名键覆盖左侧，未覆盖字段沿用 valid_user，且原基准字典不会被修改。
    cases: list[Any] = [
        ("合法记录", valid_user),
        # True 会覆盖 id，用来确认两套校验都识别 Python 中 bool 是 int 子类这个陷阱。
        ("bool 冒充 id", valid_user | {"id": True}),
        ("字符串冒充 age", valid_user | {"age": "28"}),
        # role 没有覆盖旧键，而是形成第六个键，用来对照精确字段集合/extra="forbid"。
        ("未知字段", valid_user | {"role": "admin"}),
    ]

    # 每轮把同一 payload 依次交给手写函数和 Pydantic，两个结果仅转成展示文字，
    # 从而可以检查迁移实现后“接受/拒绝”的业务决定是否一致。
    for label, payload in cases:
        try:
            manual_validate_user(payload)
            manual_result = "通过"
        except (TypeError, ValueError) as error:
            # 手写实现按规则主动抛这两类异常；圆括号表示捕获其中任意一种。
            manual_result: str = f"拒绝({error})"

        try:
            # model_validate 接收已经存在的 Python dict；这里不需要先转成 JSON 文本。
            UserModel.model_validate(payload)
```

```python
def demonstrate_manual_and_model_validation() -> None:
        try:
            # model_validate 接收已经存在的 Python dict；这里不需要先转成 JSON 文本。
            UserModel.model_validate(payload)
            model_result = "通过"
        except ValidationError as error:
            # Pydantic 把字段/模型失败统一汇总为 ValidationError，保留路径后再拼成一行。
            model_result: str = "拒绝(" + " | ".join(format_validation_error(error)) + ")"
        # 本轮两套校验都完成后并排打印；一个反例失败不会阻止后续场景继续运行。
        print(f"{label}: 手写={manual_result}; 模型={model_result}")


def demonstrate_serialization() -> None:
    """对比 Python 模式字典、JSON 模式字典和 JSON 文本。"""
    print_section("4. model_dump 的 Python/JSON 模式与 model_dump_json")

    # model_validate 接收 Python dict。嵌套 filters 也先是 dict，其中时间还是字符串；
    # 成功后 args 是 SearchArgs，args.filters 是 SearchFilters，时间字段已解析成 datetime。
    args: SearchArgs = SearchArgs.model_validate(
        {
            # query 首尾有空格，方便确认字段 validator 的返回值才是模型最终保存的内容。
            "query": "  Python Agent  ",
            # 指定 published_after 会触发跨字段规则，因此 sort 必须同时为 newest。
            "sort": "newest",
            "filters": {"published_after": "2026-01-01T00:00:00Z"},
        }
    )
    # 三种 API 的输出层级相同，但容器/时间值所处的表示阶段不同：
    # 1) Python 模式是 dict，datetime 仍可供 Python 计算；
    # 2) JSON 模式仍是 dict，但嵌套时间已变成 JSON 兼容 str；
    # 3) model_dump_json 直接返回完成编码的 JSON str。
    python_dump: dict[str, Any] = args.model_dump()
    json_mode_dump: dict[str, Any] = args.model_dump(mode="json")
    json_text: str = args.model_dump_json()

    # 连续 [] 先从外层 dict 取得 filters 子字典，再取得其中的 published_after。
    python_time: Any = python_dump["filters"]["published_after"]
```

```python
def demonstrate_serialization() -> None:

    # 连续 [] 先从外层 dict 取得 filters 子字典，再取得其中的 published_after。
    python_time: Any = python_dump["filters"]["published_after"]
    json_time: Any = json_mode_dump["filters"]["published_after"]
    # repr 会把字符串边界显示出来，因此能直接看到 query 首尾空白已被移除。
    print("validator 返回值真正保存, query:", repr(args.query))
    # 这三行同时输出实际类型和值，避免把“Python 字典”和“JSON 文本”混为一谈。
    print("Python 模式时间:", type(python_time).__name__, python_time)
    print("JSON 模式字典时间:", type(json_time).__name__, json_time)
    print("model_dump_json 返回:", type(json_text).__name__, json_text)


def demonstrate_pipeline_schema_and_errors() -> None:
    """运行完整工具数据流，并展示 schema 摘要与嵌套错误路径。"""
    print_section("5. 原始 JSON → 校验模型 → 工具 → 可序列化结果")

    # 先用普通 Python dict 组织示例输入，再由 json.dumps 编码成真正的 JSON str，
    # 模拟 Agent/tool 框架传来的原始参数文本，而不是直接把可信字典交给模型。
    raw_json: str = json.dumps(
        {
            "query": "Python",
            "limit": 2,
            "include_snippets": True,
            "sort": "newest",
            "filters": {
                "language": "zh",
                "published_after": "2026-01-01T00:00:00Z",
                "keywords": ["agent", "tool"],
            },
        },
        # 保留中文本身，不转义为 \uXXXX；这只影响文本表现，不影响解码后的值。
        ensure_ascii=False,
    )
    # 内部状态在边界输入之外单独创建；run_search 会按 validated -> completed 更新它。
    state = RunState(run_id="run-pipeline")
    result: ToolResult = run_search(raw_json, state)
```

```
533  def demonstrate_pipeline_schema_and_errors() -> None:
555      state = RunState(run_id="run-pipeline")
556      result: ToolResult = run_search(raw_json, state)
557      # state 展示内部副作用，result 则是已经校验的工具输出模型。
558      print("内部状态:", state.phase, state.events)
559      print("可发送的 JSON 结果:")
560      # indent=2 只为终端可读性缩进；返回值仍是可传输的 JSON 字符串。
561      print(result.model_dump_json(indent=2))
562
563      # model_json_schema() 返回普通 Python dict，用声明描述调用者应生成的 tool 参数形状。
564      schema: dict[str, Any] = SearchArgs.model_json_schema()
565      # properties 也是嵌套字典；按键取出 limit 子 schema，查看默认值、类型和上下界。
566      limit_schema: Any = schema["properties"]["limit"]
567      # get 在键缺失时返回 None/{}，适合只做观察，不让展示代码因可选 schema 键中断。
568      print("tool schema 必填字段:", schema.get("required"))
569      print("tool schema 的 limit:", limit_schema)
570      # $defs 保存可复用的嵌套模型定义；sorted 让展示顺序稳定。
571      print("tool schema 包含嵌套定义:", sorted(schema.get("$defs", {})))
572
573      # 这份 JSON 同时放入多种错误：严格整数、Literal、时间解析、列表元素类型，
574      # 以及内外两层未知字段，用来证明 ValidationError 能一次保留多个嵌套位置。
575      invalid_nested_json: str = json.dumps(
576          {
577              "query": "Python",
578              "limit": True,
579              "filters": {
580                  "language": "fr",
581                  "published_after": "昨天",
582                  "keywords": ["agent", 7],
583                  "unknown_filter": "x",
584              },
585              "unknown_argument": "x",
586          },
587          ensure_ascii=False,
588      )
589      try:
590          SearchArgs.model_validate_json(invalid_nested_json)
```

```
533  def demonstrate_pipeline_schema_and_errors() -> None:
589      try:
590          SearchArgs.model_validate_json(invalid_nested_json)
591      except ValidationError as error:
592          # 捕获整次边界失败后逐条输出；format_validation_error 已把 loc 转成点分路径。
593          print("嵌套错误逐项定位:")
594          for line in format_validation_error(error):
595              print(" -", line)
596
597      # 字段类型都合法不代表字段组合合理：ok=True 与非空 error 会触发
598      # ToolResult 的 after validator。
599      try:
600          ToolResult(ok=True, error="不应同时出现")
601      except ValidationError as error:
602          print("跨字段不变量错误:", " | ".join(format_validation_error(error)))
603
604      # Schema 能描述已声明的结构、类型和一部分约束，却不会执行搜索、鉴权或查数据库，
605      # 也无法判断这个本地标题/URL 是否对应现实世界的真实事实。
606      print("边界提醒：schema/校验保证已声明的结构与规则，不证明搜索内容事实正确")
607
608
609  def main() -> None:
610      """依次运行全部确定性实验。"""
611      # 函数定义本身只建立可调用对象；真正调用从这里开始，并按源码顺序产生五段输出。
612      demonstrate_data_structure_choices()
613      demonstrate_presence_and_strictness()
614      demonstrate_manual_and_model_validation()
615      demonstrate_serialization()
616      demonstrate_pipeline_schema_and_errors()
617
618
619  # 模块被直接作为脚本执行时，__name__ 等于 "__main__"，于是调用入口；
620  # 如果别的模块只是 import 本文件，则上面的类/函数会被定义，但五组演示不会自动运行。
621  if __name__ == "__main__":
622      main()
623
```

python-quickstart-for-java

example.py .../14_data_models | requirements.txt | example.py .../15_protocols_generics | example.py .../16_decorators

java_to_python > chapters > 14_data_models > requirements.txt

```
1  # 第 14 章运行依赖。2.13 系列支持 Python 3.14，并使用稳定的 Pydantic v2 API。
2  pydantic>=2.13.5,<3
3
```

- java_to_python
  - chapters
    - 09_modules
      - tax.py
    - 10_files
      - Chapter10Files.java
      - example.py
      - README.md
    - 11_environments_packages
      - demo_project
        - .python-version
        - main.py
        - pyproject.toml
        - requirements.txt
        - settings.env.example
        - uv.lock
      - .gitignore
      - example.py
      - import_lab.py
      - README.md
    - 12_http_json
      - __pycache__
      - example.py
      - local_http_fixture.py
      - README.md
      - requirements.txt
    - 13_typing
      - example.py
      - README.md
      - static_errors.py
    - 14_data_models
      - example.py
      - readme.md
      - requirements.txt
    - 15_protocols_generics
      - example.py
      - README.md
      - search_fixture.json
      - static_errors.py

~/vscode/python-quickstart-for-java/java_to_python/chapters/14_data_models/readme.md

大纲

时间线

Java Projects